InferX

Single-GPU LLM Inference Deployment and Optimization

Wenxin Chen · Zhixiong Zhang · Kunchang Guo

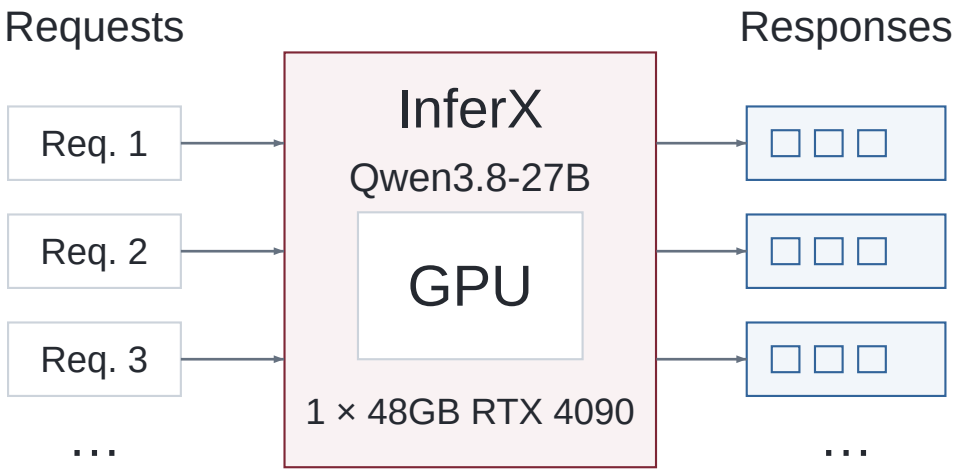
Team Ni Shuo Sha Dou Dui

Project Defense · Track A · 2026-10-18

github.com/Yuer-yuan/InferX

Project overview

Goal: higher single-GPU concurrency within latency SLOs.



First-token and generation latency SLOs

TTFT P95 ≤ 10s · TPOT P95 ≤ 40ms

Deliverable: a NInfer-based inference service

1. **4-bit computation**

   INT4 matrix kernels for weights and activations.

2. **Concurrent MTP integration**

   Expand batching; commit and roll back each request's state independently.

3. **Progress-aware scheduling**

   Chunk long inputs; use progress and urgency to allocate decode service.

Result: Supports 14 concurrent requests; both tests meet SLOs at 306.04–309.95 tok/s.

Quality: First 100 IFBench prompts: 85 strict passes and 88 loose passes.

Problems: two bottlenecks in concurrent inference

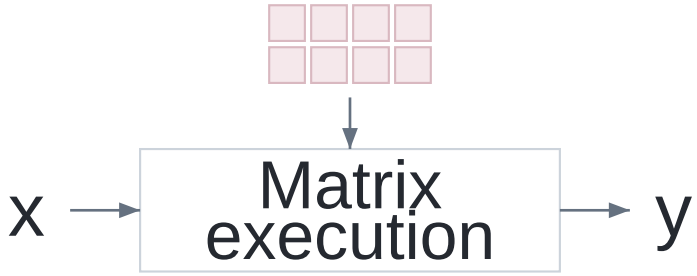
I P/D interference



Decode waits

Shared GPU execution time

II Operator efficiency

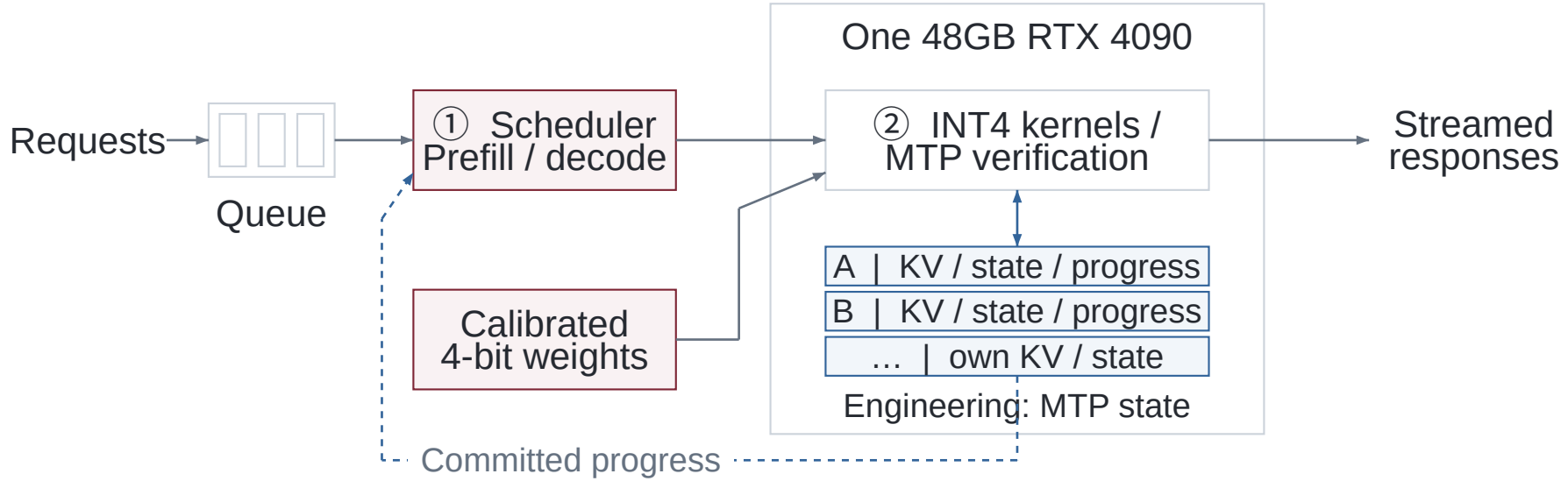


Compute and weight-access cost

- **I · P/D interference.** Long prefill work delays decode on the shared GPU; scheduling affects tail latency.
- **II · Operator efficiency.** Prefill and batched verification use different matrix shapes; low-bit gains depend on execution and conversion costs.

Goal: increase concurrency within TTFT P95 ≤ 10s and TPOT P95 ≤ 40ms; check task quality.

Design: scheduling, INT4, and concurrent MTP integration



Technical route

Given model: Qwen3.8-27B

- **Offline:** GPTQ → 4-bit weights.
- **Online:** Chunking → INT4 / MTP; Per-request commit and feedback.

Framework choice: hardware fit and integration

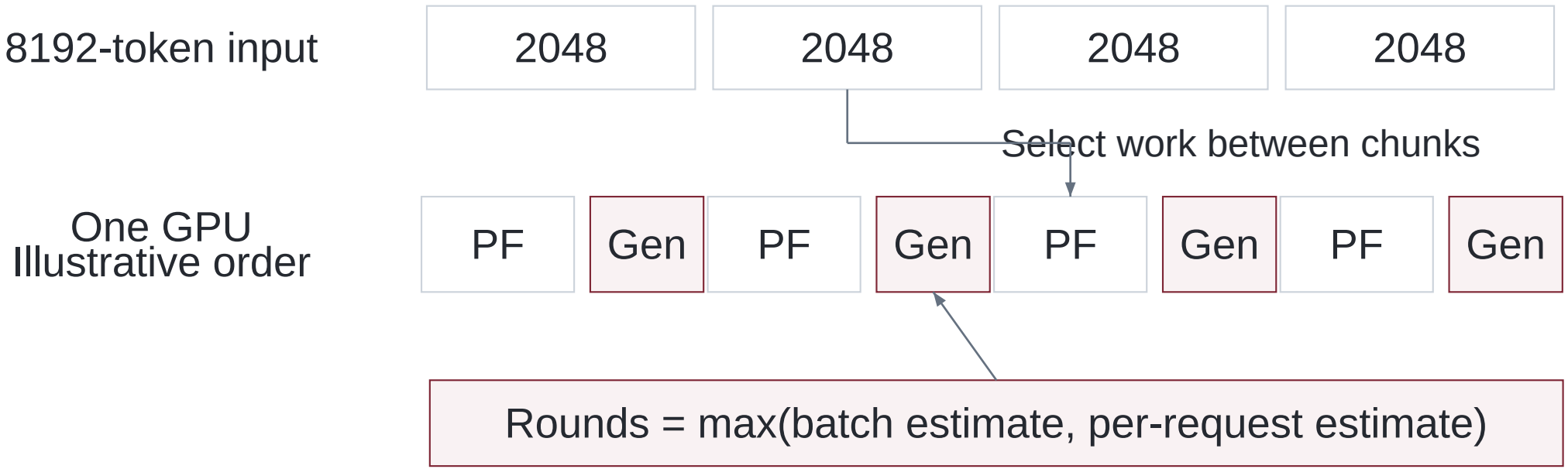
Framework	Rationale and trade-off
NInfer (chosen)	4090 / SM89 path; reuse quantization and MTP; Joint kernel/state/scheduler edits; own upkeep.
vLLM / SGLang	Support MTP for compatible models; Our W4A4 and state changes need porting.

Why ninfer-4090 fits this project

Against vLLM / SGLang, ninfer-4090 offers a focused base for single-GPU optimization.

Advantage	ninfer-4090 foundation	Value to this project
Model–GPU specialization	Qwen3.8-27B on one RTX 4090; native SM89 CUDA execution.	Tune for fixed model and matrix shapes; focus on inference on one 4090.
Low-bit path reuse	Q4/Q5 + quantized KV + MTP; reuse model loading and verification.	Add INT4 compute to an existing stack; reduce porting of our custom paths.
Execution / state control	C++20 / CUDA execution control; ReplaySSM commit / rollback.	Joint kernel, MTP, and scheduler edits; extend state, workspace, and graphs.

Design I: chunking and request progress



- **Scheduling points:** Split 8192 input tokens into four 2048-token chunks; choose prefill or decode between non-preemptible chunks.
- **Remaining work:** Track committed tokens and decode rounds per request; batch-average yield can underestimate slow requests.
- **Decision rule:** Multiply remaining rounds by current batch round time; compare deadline slack while preserving the prefill deadline guard.

Design I: estimate work from committed progress

Estimate each request separately

$$y_i = \frac{g_i - 1}{d_i}$$

$$R_i = \max\left(\left\lceil \frac{r_i}{\max(1, y_b)} \right\rceil, \left\lceil \frac{r_i}{\max(1, y_i)} \right\rceil\right)$$

g: committed tokens; d: decode rounds
r: remaining tokens; y: yield per round
i: individual request; b: current batch
Subtract one to exclude the prefill token.

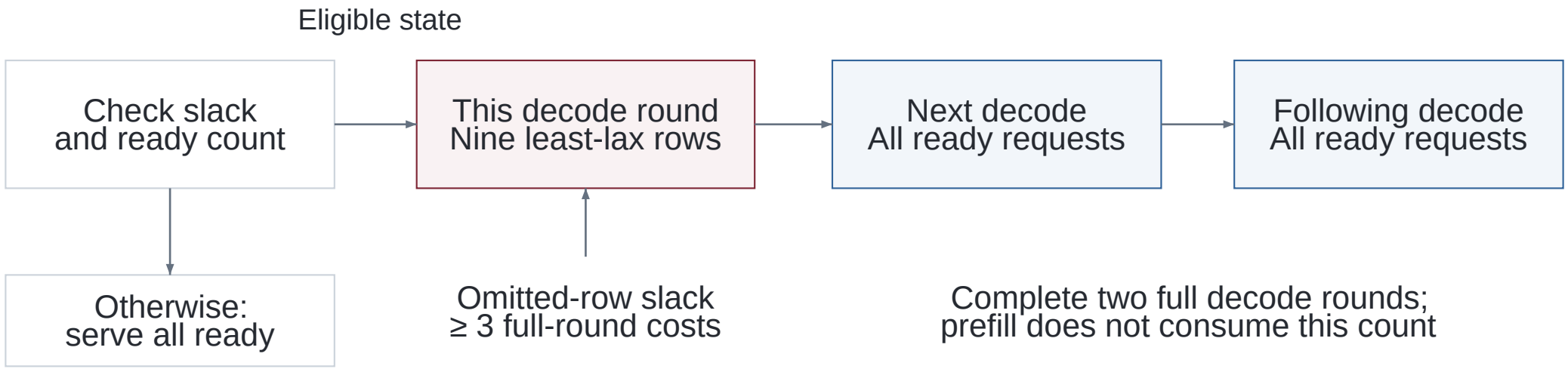
- **Update after commit.** Use the output policy's actual commit count; cancelled rows do not count as rounds.
- **Cold-start fallback.** Use the existing batch estimate before any decode history is available.
- **Keep the batch lower bound.** Slow requests may increase predicted work; fast requests never reduce the original estimate.

Matched C16; estimator change only; 128 measured + 8 warmup

Estimator	TTFT P95 (s)	TPOT P95 (ms)	tok/s
Batch estimate	9.856	48.784	311.77
With request progress	9.838	45.745	315.73

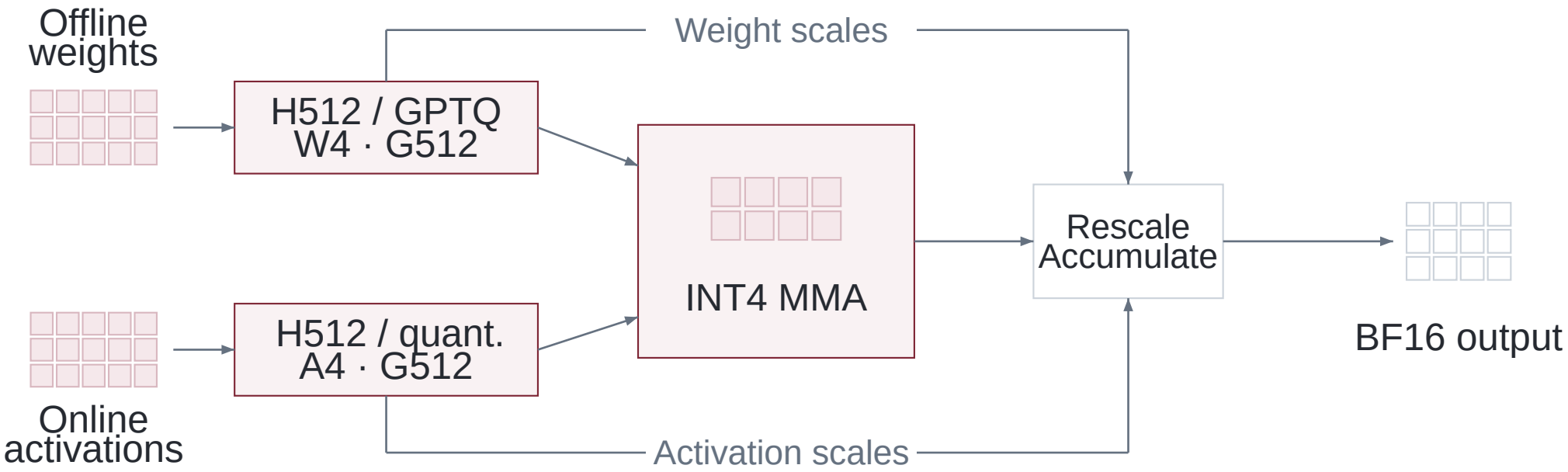
TPOT improves 6.23%, still above 40 ms; model operators and CUDA Graph are unchanged.

Design I: bounded urgent service, then two full rounds



- Check only after the original scheduler chooses decode: 10–14 ready requests, a negative slack, and sufficient slack for every omitted row.
- At most one subset round at a time; the next two actual decode rounds serve all ready requests. Prefill does not consume this count.
- Count progress only for served rows. This redistributes service; evaluate aggregate throughput and SLOs on the complete service.

Design II: INT4 weights and activations



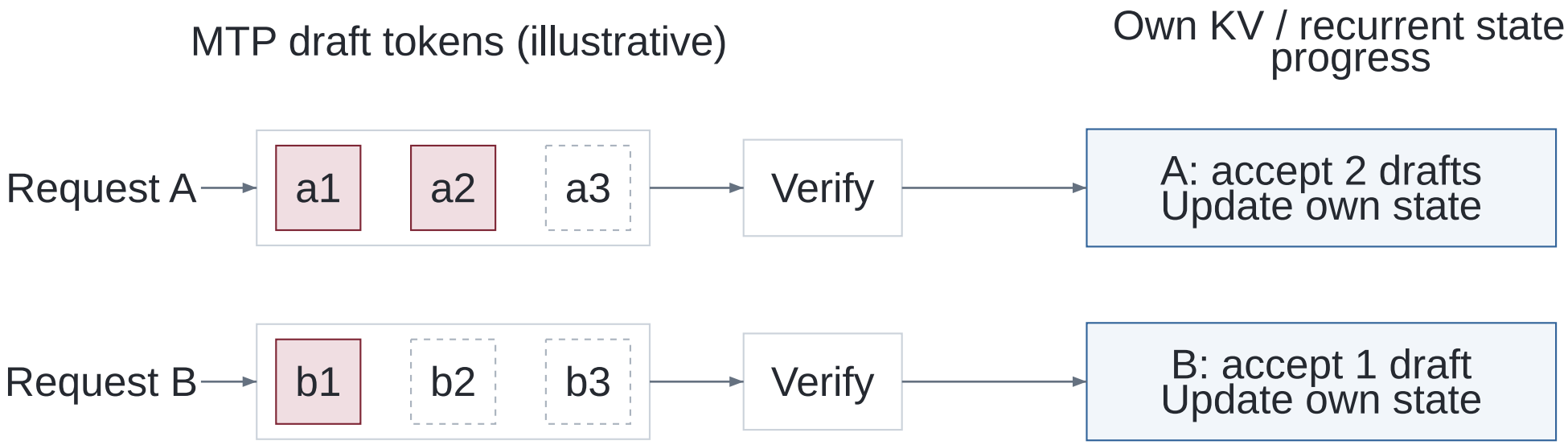
- **Offline weights:** Apply local 512-dimensional Hadamard transforms and GPTQ in 512-channel groups; cache 4-bit codes and scales.
- **Online activations:** Transform BF16 activations and dynamically quantize each group; feed both operands to S4 matrix instructions.
- **Output reconstruction:** Accumulate INT32 within groups and scaled FP32 across groups; output BF16 while preserving nonlinear and residual semantics.

Design II: integrate quantized paths into the model

Component	Implementation	Validation
Quantized caches	Start with 128 FFN matrices across 64 layers; use four 8192-token calibration requests and two held out.	Check codes, scales, and held-out error.
Kernels and routing	Extend FFN paths to input/output projections; connect real weights, workspace, and Graph dependencies.	Time transforms, quantization, matrix work, and writeback together.
MTP verification	Enable T33–128 in 47 GDN output layers; keep prior routes for T1–32, L0, and full-attention outputs.	T is the token-row count; verify coverage by layer, phase, and shape.

- **Our implementation:** Build on existing GPTQ and Hadamard methods; implement caches, CUDA operators, and model routing.

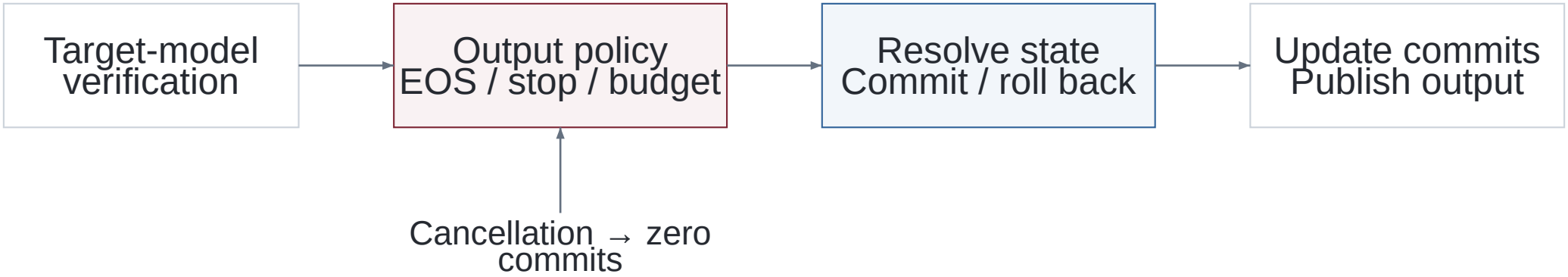
Implementation: adapting MTP for concurrency



Solid: accepted; dashed: rejected. Drafts only; fallback omitted. Accepted drafts ≠ total commits.

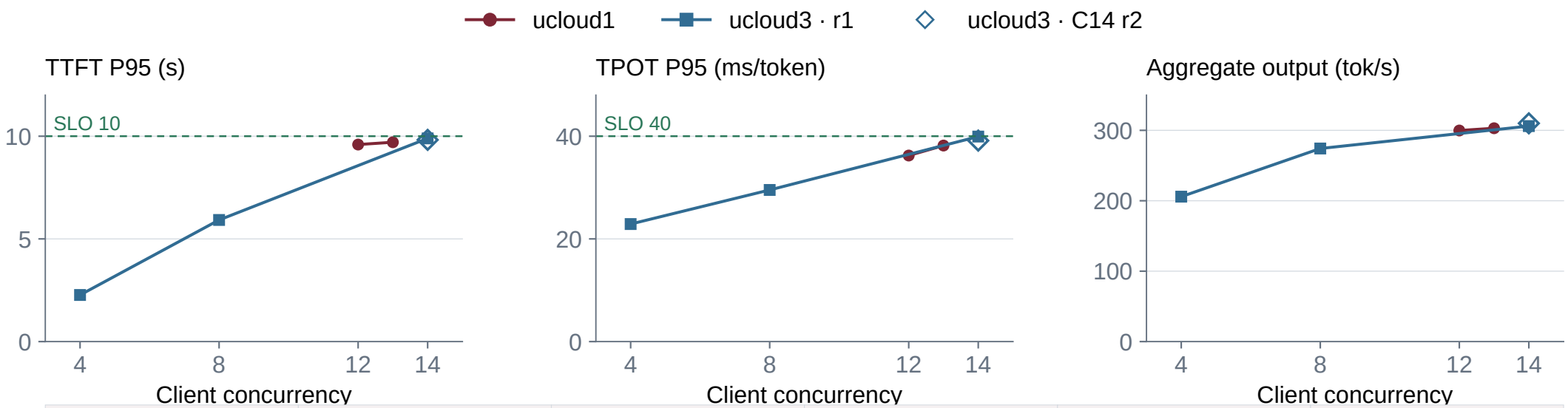
- **Operators and layouts:** Extend attention/GDN verification shapes, row-to-slot mapping, I/O buffers, and state records for 16/32 requests.
- **Runtime resources:** Size KV, state, and workspace for configured concurrency; update CUDA Graph arguments and buffer bounds.
- **State invariant:** Maintain separate KV, GDN recurrent, and convolution states; unequal acceptance must still match each committed sequence.

Implementation: per-request state commit and rollback



- **Resolve output first:** After target verification, EOS/stop and remaining budget determine actual commit lengths; cancelled rows commit zero.
- **Update per request:** Resolve pending state, then update committed tokens, generation budget, and decode rounds; publish valid output last.
- **Validation coverage:** Check high slots, mixed 0–4 commits, cancel/reset, and Graph replay. The state-fusion stage has 2013 matching records; 86 tests pass, four skip.

Results: latency and throughput under concurrent load



Node / run	Client / native	KV tokens	TTFT P95 (s)	TPOT P95 (ms)	tok/s
ucloud3 · r1	4 / 4	65536	2.269	22.891	205.82
ucloud3 · r1	8 / 8	69632	5.922	29.532	274.20
ucloud1 · r1	12 / 12	104448	9.594	36.235	299.76
ucloud1 · r1	13 / 13	113152	9.706	38.181	302.97
ucloud3 · r1	14 / 13	121856	9.894	39.929	306.04
ucloud3 · r2	14 / 13	121856	9.822	39.142	309.95

8192 input / 512 output tokens; 128 measured requests and 8 warmups per setting.

Each curve represents one node. Two tests at concurrency 14; TTFT includes queueing.

Results: passing configurations and capacity limits

Concurrency 14 meets both SLOs twice; 15 and 16 exceed the TPOT limit.

Node / run	Client / native	TTFT P95 (s)	TPOT P95 (ms)	tok/s	SLO
ucloud1 · r1	14 / 14	9.725	41.022	306.06	FAIL
ucloud3 · r1	14 / 13	9.894	39.929	306.04	PASS
ucloud3 · r2	14 / 13	9.822	39.142	309.95	PASS
ucloud1 · r1	15 / 13	9.96	41.881	309.66	FAIL
ucloud3 · r1	16 / 16	9.967	51.221	316.74	FAIL

- Passing configuration: admit at most 13 active requests; others wait, with queueing included in TTFT.
- Targets: TTFT P95 ≤ 10 s and TPOT P95 ≤ 40 ms.

Results: instruction following on the first 100 IFBench prompts

100 prompts, 111 instruction constraints

Score	Result
Prompt strict	85 / 100
Prompt loose	88 / 100
Instruction strict	96 / 111
Instruction loose	99 / 111

- Scope: the first 100 IFBench prompts selected by the original evaluation script, scored with its original verifier.
- Strict prompt scoring: a prompt passes only when every instruction constraint is satisfied.
- Setup: thinking enabled, up to 32768 output tokens; the same binary as the performance tests.

Quality cases: what passes and what fails

Actual responses from the first 100 IFBench prompts

PASS · key80

Prompt requirements

Imagine Arya disguising others as herself. Write three equal-length sentences using distinct words; indent each line further.

Actual answer (complete)

```
She swapped faces with ease.
    Her mimicry took all shapes.
        Every mask looked like hers.
```

- 0 / 4 / 8 leading spaces; 28 characters per sentence after trimming; 15 distinct words.
- Both instructions pass; prompt strict score = 1.

FAIL · key88

Prompt requirements

Give the range of n for the stated flow problem. Use !?!? as the list marker; adjacent words must start with different letters.

Actual answer (complete)

```
!?? 0 < n < 1
!?? valid range
```

- Expected !?!?; both items use !??, missing one character from the required marker.
- Only one of two instructions passes, so the prompt fails.

Engineering: obtain and verify

① Obtain the reproduction bundle and prepare a fresh directory

```
export INFERX_REPRO=/workspace/InferX/reproductions/rerun01
mkdir -p /workspace/InferX/reproductions
mkdir "$INFERX_REPRO"
tar -xzf reproduction-kit-v1.tar.gz \
  -C "$INFERX_REPRO" --strip-components=1
cd "$INFERX_REPRO"
sha256sum -c SHA256SUMS
```

Input	Fixed identity	Checks in this run
Complete engine source	1023 files · c976692d…	Archive and per-file SHA; checked before/ after build
Model and quantized caches	Pinned model revision; 025 / 041 / 047	Rehashed the model and all 478 serving files

Run from the bundle directory. u3 tests reuse existing caches; u2 regenerates and validates separate assets. See the bilingual guide.

Engineering: regenerate quantized caches

② Rebuild three cache families from FP8 and fixed calibration inputs

```
python3 "$INFERX_Q/reproduce.py" \
  --work-dir "$INFERX_W" --fp8 "$INFERX_FP8" \
  --model "$INFERX_M" --tokenizer "$INFERX_T" \
  --fit-python "$FIT_PY" --build-env "$INFERX_B" \
  --cuda-root "$INFERX_C" --xgrammar "$INFERX_X" \
  --eval-envs "$INFERX_E" --tools "$INFERX_U"
```

Stage	Fresh capture and fitting	Outputs
025	Base Q4/Q5 forward capture → FFN fit	64 gate/down layers · 256 files
041	Capture with new025 → input projection fit	64 input projections · 128 files
047	Capture with new025 + 041 → output fit	47 selected output projections · 94 files

Four fixed training and two held-out inputs; per-file FP8 revisions/hashes and a separately acquired base .ninfer model.

Set paths as in the guide; run 116 stages and final receipts are separate. The following 664 tests use old caches.

Engineering: build from source

③ Build normally and bind the new executable

```
python3 build_owner.py
python3 bind_rebuilt_recipe.py
python3 -B -m unittest discover \
  -s entrypoint/tests -p "test_*.py" -v
```

- Release / SM89 · CUDA13.2.51 / GCC13.4; a full build in a fresh directory.
- All 1023 source files match before/after build; 7 scheduler CPU tests pass, with 55 runtime dependencies recorded.
- Archive the executable with its runtime configuration; use this build for the following performance and quality reproduction.

Inspect build-job.json, runtime-identity.json and rebuilt-recipe-binding.json.

Engineering: start the service

④ Start the performance service in the new deployment

```
python3 -B entrypoint/serve_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --profile perf-c14 --root "$PWD" --port 18540 \
  --log-dir "$PWD/service-perf-c14-001"
```

- Client 14 / native 13; ordinary MTP K3, KV121856, context 65536 and prefill chunk 2048.
- Server and client both select the rebuilt recipe; the 19 environment flags retain the frozen 100 configuration.
- Service logs and launch.json record the live process, executable, paths and arguments.

Engineering: reproduce performance

⑤ Run the original benchmark in another local terminal

```
OPENAI_API_KEY=EMPTY python3 -B entrypoint/perf_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --concurrency 14 --root "$PWD" \
  --service-launch "$PWD/service-perf-c14-001/launch.json" \
  --base-url http://127.0.0.1:18540/v1 \
  --output-dir "$PWD/performance-C14-001"
cat performance-C14-001/summary.json
```

- 128 measured + 8 warmup + 1 probe; cold 8192 / output 512, retaining initial measured requests.
- Verify the live service and all 137 payloads; raw records are in performance-C14-001/c14.

Engineering: reproduced performance

⑥ Original JSON excerpt from this reproduction

ucloud3 · new executable 664cfbb9… · client 14 / native 13 · latency in ms


```
{
  "concurrency": 14,
  "records": 128,
  "error_requests": 0,
  "metrics": {
    "time_to_first_token": {"p95": 9896.41735465},
    "inter_token_latency": {"p95": 38.285849954403 13}
  },
  "output_token_throughput": 304.89665937584215
}
```


TTFT 9.896 s and TPOT 38.286 ms pass both SLOs in this run; throughput 304.90 tok/s.

Engineering: reproduce quality

⑦ Stop the performance service, then start quality

```
python3 -B entrypoint/serve_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --profile quality100 --root "$PWD" --port 18541 \
  --log-dir "$PWD/service-quality100-002"
```

Evaluate in another local terminal

```
OPENAI_API_KEY=EMPTY python3 -B entrypoint/accuracy_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --root "$PWD" \
  --service-launch "$PWD/service-quality100-002/launch.json" \
  --base-url http://127.0.0.1:18541/v1 \
  --output-dir "$PWD/quality100-002"
```

Client/native 13 · KV532480 · max32768 · natural EOS, default thinking and sampling.

Verify process identity and wait for model readiness; quality evidence: quality100-002/evaluation/accuracy.

⑧ Reproduction scores on the first 100 IFBench prompts

New executable 664cfbb9… · C13 / KV532480 · scores are proportions


```
{
  "accuracy": {
    "status": "完整",
    "generated_samples": 100,
    "metrics": {
      "prompt_level_strict": {"score": 0.9, "num": 100},
      "inst_level_strict": {"score": 0.9009, "num": 111},
      "prompt_level_loose": {"score": 0.96, "num": 100},
      "inst_level_loose": {"score": 0.955, "num": 111}
    }
  }
}
```

Engineering: exception handling

Failure	Response	Retained evidence
Startup failure / port in use	Check before launch; bound the readiness wait.	Startup log, exit code and actual config.
Out of memory	Inspect KV, active capacity and Graph; retest a revised config.	OOM log and original parameters.
Invalid request	Reject an invalid model ID or token with HTTP 400.	Request, status code and error response.
Timeout / empty / truncated output	Keep failures and partial output; score by the original rules.	Client record, finish reason and output length.

Engineering: reliability

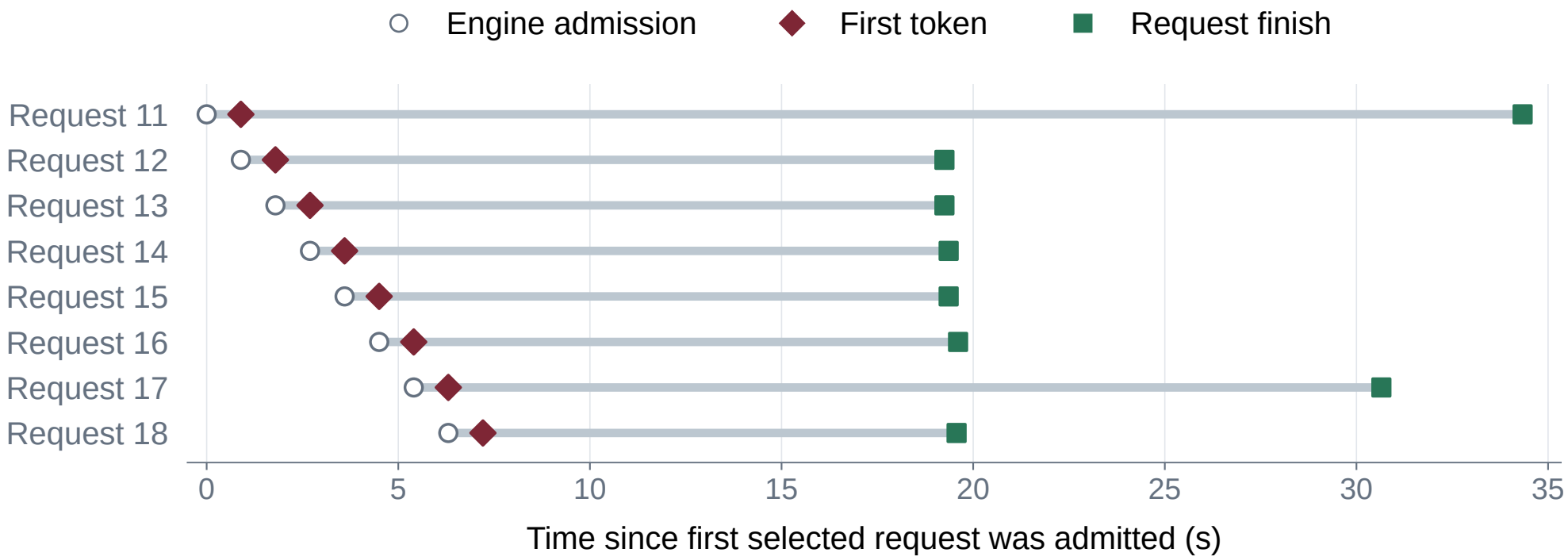
Scope	Invariant	Verification
API output	Consistent stream termination, token counts and finish reasons.	Probe streamed/non-streamed output, usage and invalid inputs.
Concurrent state	Independent commit and rollback; slot reuse preserves isolation.	High slots, partial commits, cancel/reset and Graph replay.
Service process	Verify process identity before cleanup.	Check PID, start time and executable; clean up via pidfd.

State-fusion experiment: 2013 matching records; 86 tests passed, 4 skipped.

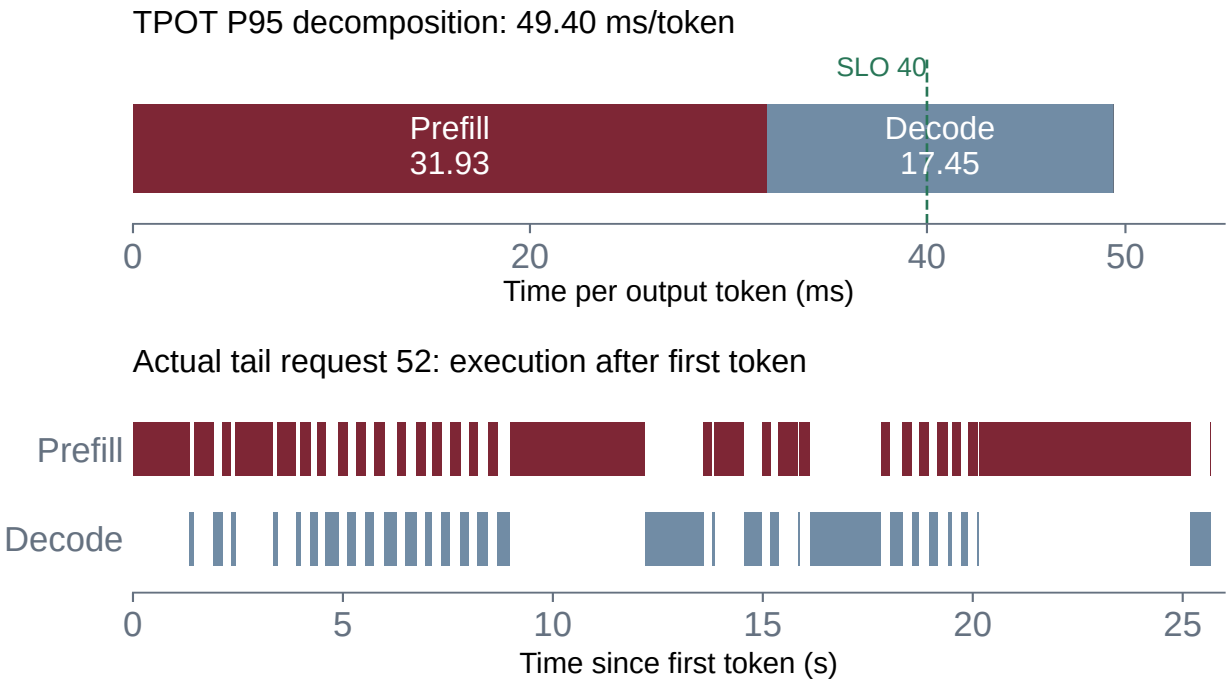
Engineering: observability

Real request traces: admission → first token → finish

085 / C16 · First 8 measured requests, ordered by engine admission

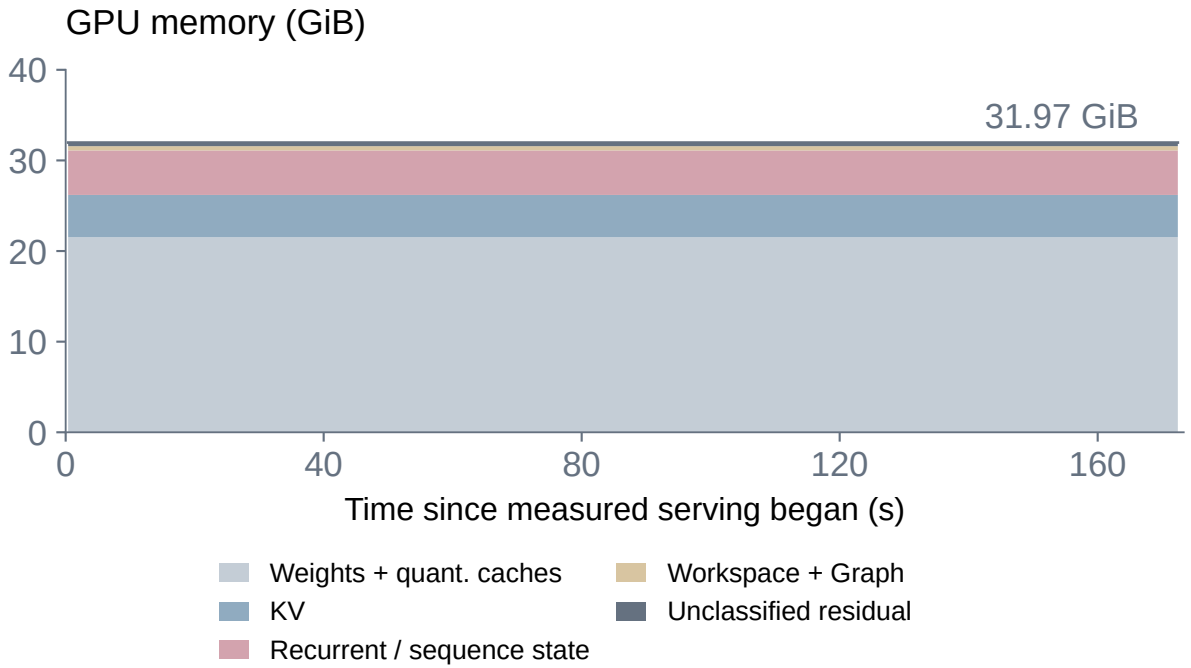


Bottleneck: prefill delays decode service



- Competing prefill contributes 64.6% of TPOT P95: 31.93 of 49.40 ms.
- Each 8192-token input still needs a median 0.909 s of prefill execution.
- One 085 / C16 diagnostic run, 128 cold-input requests; original P95 contributors.
- Synchronized executor wall time measures service interference, not exclusive GPU time.

Resources: GPU memory during actual serving



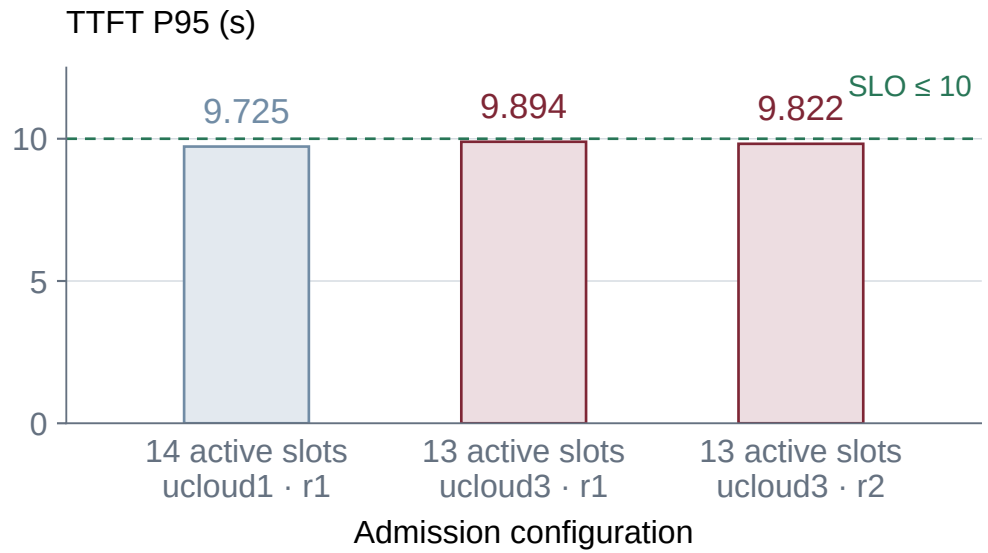
Memory breakdown (GiB)

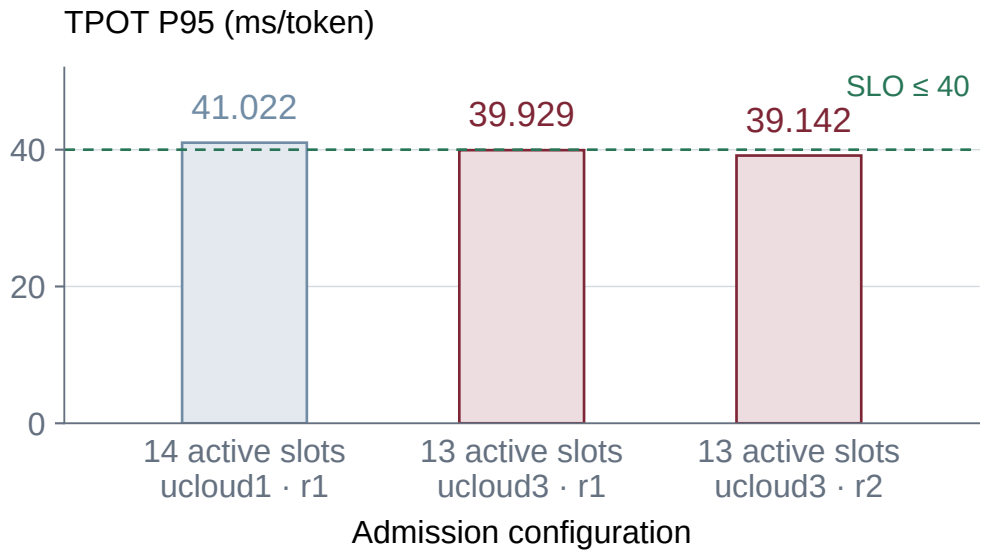
Weights + quant. caches	21.53
KV cache	4.66
Recurrent / sequence state	4.91
Workspace + Graph	0.47
Unclassified residual	0.40

- Same 085 run: 173 samples over 173 s, all at 31.97 GiB.
- Startup allocations explain components; device samples give totals. Graph uses 138 MiB.

Tradeoff: spend TTFT headroom to limit contention

14 concurrent client requests; compare two active-request limits.





- Mechanism and cost: admit at most 13 active requests; others wait. Queueing counts toward TTFT and limits active contention.
- Choice: 14 slots miss TPOT; 13 slots meet both SLOs twice at 306.04–309.95 tok/s.

Conclusions and limitations

INT4 execution → **Request state + scheduling** → Latency-constrained capacity

- Delivered INT4 inference, progress-aware scheduling and concurrent MTP; both tests at concurrency 14 meet SLOs.
- First 100 IFBench prompts: 85 strict passes and 88 loose passes.
- Remaining bottleneck: prefill still delays decode; concurrency 15/16 exceeds the TPOT limit.
- Next: improve prefill kernels and scheduling, repeat load tests, and evaluate the full dataset.